

- ✓ 사이킷런으로 로지스틱 회귀 모델 프로세스 따라가기
- ✓ 필요한 라이브러리 불러오기

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

numpy

- 숫자와 배열 데이터를 처리하기 위한 라이브러리
- 머신러닝에서는 입력 데이터를 배열 형태로 다루기 때문에 자주 사용함

matplotlib.pyplot

- 데이터를 그래프로 표현하기 위한 라이브러리
- 여기서는 데이터 분포와 결정 경계를 시각화할 때 사용함

LogisticRegression

- scikit-learn에서 제공하는 로지스틱 회귀 분류 모델
- 입력 데이터를 보고 클래스 0 또는 클래스 1을 예측함

train_test_split

- 데이터를 훈련용 데이터와 테스트용 데이터로 나누는 함수

accuracy_score

- 모델의 예측값과 실제값을 비교하여 정확도를 계산하는 함수

✓ 1. 데이터 만들기

```
# --- 1. 데이터 생성 (순수 Python 예시와 동일한 가상의 이진 분류 데이터) ---
np.random.seed(42) # 재현성을 위해 시드 설정
# 다시 실행해도 동일한 랜덤 데이터가 만들어지도록 하기 위함

# 클래스 0 데이터
X0 = np.random.randn(50, 2) * 0.8 + np.array([-1, -1])
y0 = np.zeros(50)

# 클래스 1 데이터
X1 = np.random.randn(50, 2) * 0.8 + np.array([1, 1])
y1 = np.ones(50)

X = np.vstack((X0, X1)) # 특성 데이터 합치기
y = np.hstack((y0, y1)) # 타겟 데이터 합치기 (scikit-learn은 1D array도 잘 처리)
```

```
print("원본 데이터 x 형태:", X.shape)
print("원본 데이터 y 형태:", y.shape)
```

randn 함수

- 평균이 0이고 표준편차가 1인 표준 정규분포에서 무작위 난수를 생성하는 함수

클래스 0 데이터

- 특성 2개를 가지는 랜덤 데이터 50개 생성
- 각 특성값에 연산 -1 적용
- 라벨 '0'으로 설정

클래스 1 데이터

- 특성 2개를 가지는 랜덤 데이터 50개 생성
- 각 특성값에 연산 +1 적용
- 라벨 '1'로 설정

데이터 합치기

- vstack : 배열을 세로 방향으로 연결함
- hstack : 배열을 이어붙여 하나의 배열로 만듦

✓ 2. 데이터 분할

```
# --- 2. 데이터 분할 (훈련 세트와 테스트 세트) ---
# 분류 문제이므로 stratify=y 옵션을 사용하여 클래스 비율을 유지합니다.
X_train, X_test, y_train, y_test = \
    train_test_split(X, y,
                    test_size=0.2,
                    random_state=42,
                    stratify=y)

print("\n훈련 세트 x 형태:", X_train.shape)
print("테스트 세트 x 형태:", X_test.shape)
print("훈련 세트 y 형태:", y_train.shape)
print("테스트 세트 y 형태:", y_test.shape)
```

머신러닝에서는 일반적으로 전체 데이터를 아래와 같이 나눔

- 훈련 데이터(train) : 모델을 학습시킬 데이터
- 테스트 데이터(test) : 학습한 모델의 성능을 확인할 데이터

test_size=0.2

- 전체 데이터 중 20%를 테스트 데이터로 사용한다는 의미.

stratify=y

- 훈련 데이터와 테스트 데이터에서 클래스의 비율이 유지되도록 함.

✓ 3. 모델 생성 및 훈련

```
# --- 3. 로지스틱 회귀 모델 생성 및 훈련 ---
# LogisticRegression 모델 객체를 생성합니다.
# solver='liblinear'는 최적화 방법을 지정함.
# liblinear는 비교적 작은 데이터셋에 적합하고 이진 분류에 잘 작동합니다.
# random_state는 재현성을 위해 설정합니다.
# max_iter는 수렴을 위한 최대 반복 횟수입니다. (경사 하강법의 n_iterations와 유사)
model = LogisticRegression(solver='liblinear',
                           random_state=42,
                           max_iter=1000)

# 모델 훈련 (학습)
# .fit() 메서드를 호출하면 내부적으로 경사 하강법 등을 통해 최적의 가중치를 찾습니다.
model.fit(X_train, y_train)

print("\n모델 훈련 완료!")

# 학습된 가중치(w)와 편향(b) 확인
# model.coef_는 특성별 가중치, model.intercept_는 편향(절편)입니다.
# coef_는 2차원 배열이므로 flatten()을 사용하여 보기 쉬운 1차원 배열로 변환함.
print(f"학습된 가중치 (w): {model.coef_.flatten()}")
print(f"학습된 편향 (b): {model.intercept_[0]:.4f}")
```

✓ 4. 테스트 데이터 예측

```
# --- 4. 예측 수행 및 확률 예측 ---
# 테스트 세트에 대한 클래스 예측
y_pred = model.predict(X_test)
print("\n테스트 세트 예측 클래스 (상위 5개):", y_pred[:5])
print("테스트 세트 실제 클래스 (상위 5개):", y_test[:5])

# 테스트 세트에 대한 클래스별 확률 예측
# 첫 번째 열은 클래스 0에 속할 확률, 두 번째 열은 클래스 1에 속할 확률
y_proba = model.predict_proba(X_test)
print("\n테스트 세트 클래스별 확률 (상위 5개):")
print(y_proba[:5]) # [:5]는 앞에서부터 5개의 값만 확인한다는 의미.
```

클래스 예측

- predict()에 처음 보는 테스트 데이터를 입력하고 클래스를 예측하게 함.
- 예측한 값과 실제 정답을 비교할 수 있음.

클래스별 확률 예측

- predict()는 최종 클래스만 알려주지만, predict_proba()는 각 클래스에 속할 확률을 알려줌.
- 예를 들어 [0.85, 0.15]가 나오면 클래스 0일 확률 : 85%, 클래스 1일 확률 : 15%라는 의미.

✓ 5. 모델 평가

```
# --- 5. 모델 평가 ---
# 정확도 계산
accuracy = accuracy_score(y_test, y_pred)
print(f"\n테스트 세트 정확도: {accuracy:.4f}")
```

accuracy_score(실제 정답, 모델의 예측)

- 실제 정답과 모델의 예측을 비교함.
- 결과값 = 맞게 예측한 데이터 수 / 전체 데이터 수
- 예를 들어 테스트 데이터가 20개이고 모델이 18개를 맞췄다면, $18 / 20 = 0.9$ 으로 정확도는 90%가 됨.

✓ 데이터 시각화

```
# --- 6. 결정 경계 시각화 ---
plt.figure(figsize=(10, 8))
plt.scatter(X[y == 0, 0], X[y == 0, 1],
            color='blue', label='class 0', alpha=0.7)
plt.scatter(X[y == 1, 0], X[y == 1, 1],
            color='red', label='class 1', alpha=0.7)

# 결정 경계 그리기
# 결정 경계는  $w_1x_1 + w_2x_2 + b = 0$  이 되는 지점입니다.
#  $x_2 = (-b - w_1x_1) / w_2$ 
# model.coef_[0][0]은  $w_1$ , model.coef_[0][1]은  $w_2$ 
# model.intercept_[0]은  $b$ 
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.RdBu) # 결정 영역
plt.contour(xx, yy, Z, colors='k',
            linestyle='--', levels=[0.5]) # 결정 경계 (0.5 확률)

plt.xlabel("feature 1", fontsize=14)
plt.ylabel("feature 2", fontsize=14)
plt.title("Decision Boundary", fontsize=16)
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.legend(fontsize=12)
plt.grid(True)
plt.show()
```

▷ 전체 코드

↳ 숨겨진 셀 1개

